# E0 297: Cryptographic Hardware Accelerators

# Design Assignment 1: Iterative AES-256 Core

ASIC Synthesis and Implementation Analysis Report

Anirudh Chaturvedi
SR No. 25850
M.Tech VLSI, Dept. of ECE
Indian Institute of Science, Bengaluru

September 2026

**Course Instructor:** Prof. Utsav Banerjee (Dept. of ESE, IISc)
**Target Technology:** Nangate Open Cell Library (45 nm, Typical Corner, 1.1 V, 25°C)
**CAD Tools Used:**

- Yosys 0.52 (logic synthesis, RTL elaboration, technology mapping)
- OpenSTA 3.1.0 (static timing analysis, cell-level power estimation)
- Icarus Verilog v12.0 (functional verification and testbench simulation)

## Contents

**1 Design Overview and Architectural Choices** **3**
1.1 Architectural Trade-Off: Serial vs. Round-Based Execution . . . . . 3
1.2 Top-Level Datapath and Block Diagram . . . . . 3
1.3 Round Transformations and Mathematical Details . . . . . 4
1.4 Key Schedule . . . . . 5
1.5 S-Box Implementations: LUT vs. Canright . . . . . 5
1.6 FSM Controller and Execution Timing . . . . . 5

**2 Synthesis Results** **6**
2.1 Synthesis Flow and Hierarchy Flattening . . . . . 6
2.2 Area Comparison . . . . . 6
2.3 Standard Cell Breakdown . . . . . 7

**3 Static Timing Analysis** **7**
3.1 Timing Results and Operating Frequency . . . . . 8
3.2 Critical Path Analysis . . . . . 8

**4 Power and Energy Analysis** **8**
4.1 Discussion: Leakage vs. Dynamic Power . . . . . 9

**5 Functional Verification** **9**

**6 Summary and Comparison** **10**

**7 Submission File Listing** **11**

# 1 Design Overview and Architectural Choices

In Lectures 5 and 6, we studied the hardware design space of the Advanced Encryption Standard (AES) [1, 2]. Implementation strategies range from compact 8-bit byte-serial datapaths to fully unrolled 14-round pipelines [4, 3, 7]. For this assignment, we designed and implemented an iterative **round based (parallel)** architecture for AES-256.

## 1.1 Architectural Trade-Off: Serial vs. Round-Based Execution

A common low area alternative is the byte serial datapath [4], which operates on 8 bits per clock cycle and time-multiplexes a single S-Box between the state transformation and key expansion. While a byte-serial core occupies very little silicon ($\approx 3.1\,\text{kGE}$ [4]), it requires over 200 clock cycles to complete one 14-round AES-256 block encryption. As discussed in Lecture 6, clocking the register files, multiplexers, and control state machine across hundreds of cycles burns considerable static and clock network energy (the "carpooling" concept: amortizing clock distribution over wide parallel datapaths is substantially more energy-efficient [7]).

In contrast, our round-based architecture processes the full 128-bit cipher state in parallel, computing one complete AES round per clock cycle. The core uses a total of **20 S-Box instances**:

- 16 S-Boxes operating concurrently for the `SubBytes` transformation across the $4 \times 4$ state array.
- 4 S-Boxes operating concurrently for the `SubWord` step in the on-the-fly key expansion logic.

This design delivers gigabit-per-second encryption throughput with a lean control FSM and high energy efficiency per encrypted block.

## 1.2 Top-Level Datapath and Block Diagram

Figure 1 shows the block diagram of our iterative AES-256 core. The datapath centers around two primary registers:

- `state_reg` (128-bit): registers the intermediate state at each round boundary.
- `key_reg` (256-bit): holds the rolling 256-bit key schedule, updated on each cycle.

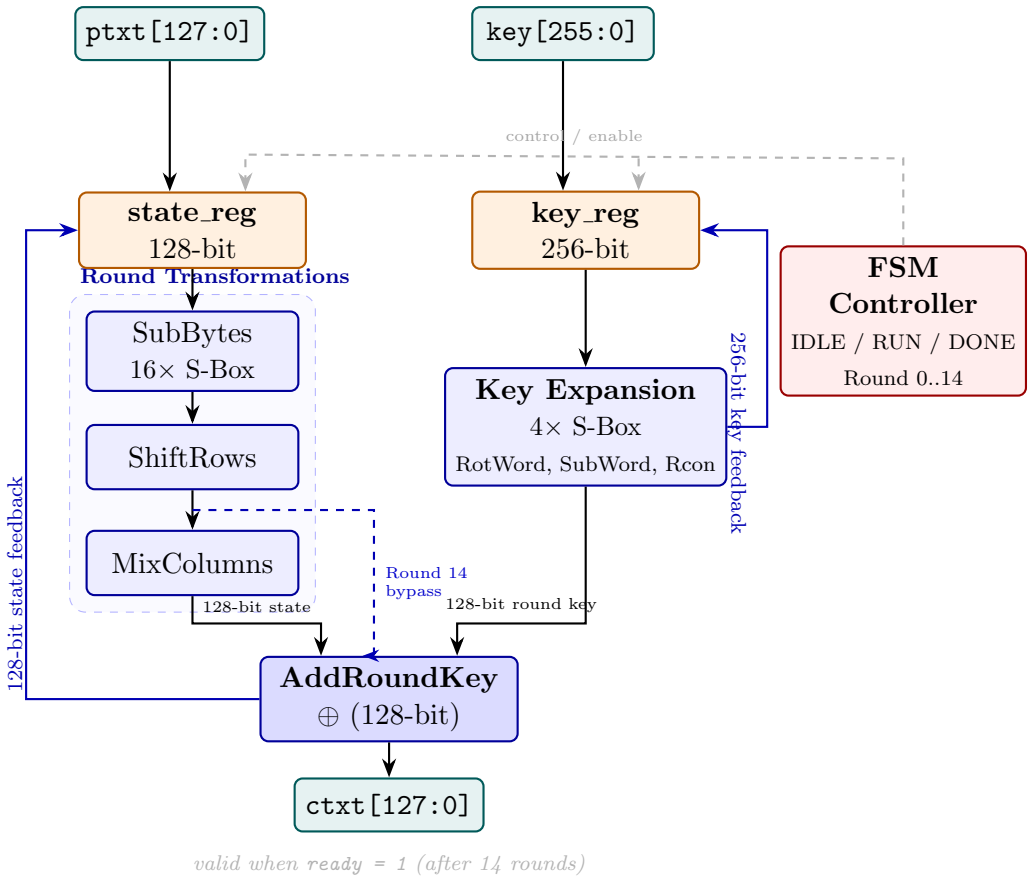



Figure 1: Block diagram of our iterative AES-256 core. Left column: round transformations (SubBytes, ShiftRows, MixColumns). Middle column: on-the-fly key schedule. Blue arrows indicate register feedback loops; dashed gray lines show FSM control.

## 1.3 Round Transformations and Mathematical Details

Following the AES mathematical formulation from Lecture 5 [1, 2]:

1. **SubBytes**: Replaces each byte $a \in \mathrm{GF}(2^8)$ independently by:

$$S(a) = \text{Affine}\left(a^{-1} \bmod m(x)\right) \tag{1}$$

where $m(x) = x^8 + x^4 + x^3 + x + 1$ is the Rijndael irreducible polynomial over GF(2). Inversion in $\mathrm{GF}(2^8)$ provides maximum non-linearity against differential and linear attacks, while the GF(2) affine mapping eliminates simple algebraic shortcuts [2].

2. **ShiftRows**: Cyclically rotates row $r$ ($r \in \{0, 1, 2, 3\}$) of the $4 \times 4$ state matrix to the left by $r$ bytes:

$$s'_{r,c} = s_{r,(c+r) \bmod 4} \tag{2}$$

In hardware, this requires zero logic gates and is realized entirely by wire routing.

3. **MixColumns**: Treats each 4-byte column as a polynomial over $\mathrm{GF}(2^8)$ and multiplies it modulo $x^4 + 1$ with $c(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} \{02\} & \{03\} & \{01\} & \{01\} \\ \{01\} & \{02\} & \{03\} & \{01\} \\ \{01\} & \{01\} & \{02\} & \{03\} \\ \{03\} & \{01\} & \{01\} & \{02\} \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \tag{3}$$

Addition is bitwise XOR. Multiplication by {02} is implemented using the `xTimes(b)` function:

$$b \bullet \{02\} = (b \ll 1) \oplus (b_7 \cdot \{1\text{B}\}_{16}) \tag{4}$$

Multiplication by {03} is evaluated as xtime($b$) $\oplus$ $b$. In round 14 (the final round), MixColumns is bypassed using a 2-to-1 multiplexer.

4. **AddRoundKey**: Computes the bitwise XOR between the 128-bit intermediate state and the 128-bit round key extracted from `key_reg`.

## 1.4 Key Schedule

AES-256 uses a 256-bit cipher key to derive 15 round keys (60 32-bit words). Rather than pre-computing and storing all round keys in a 1920-bit register array, we generate the round keys dynamically using a 256-bit rolling register. The key schedule alternates between two rules:

- **Odd rounds** ($r = 1, 3, \ldots, 13$): $w_i = w_{i-8} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[r]$
- **Even rounds** ($r = 2, 4, \ldots, 14$): $w_i = w_{i-8} \oplus \text{SubWord}(w_{i-1})$ (without RotWord or Rcon)

Four dedicated S-Box instances perform the 32-bit `SubWord` substitution concurrently with the state datapath, eliminating any key scheduling stall cycles.

## 1.5 S-Box Implementations: LUT vs. Canright

The assignment evaluates two standard S-Box implementations provided in `AES_SBOX.v`:

1. **Look-Up Table (LUT) S-Box** (`SBOX_TYPE = 0`): A direct 256-entry case statement.
2. **Canright Composite-Field S-Box** (`SBOX_TYPE = 1`): Computing inversion directly over GF($2^8$) is hardware-intensive. This replaces wide multiplexer trees with compact networks of XOR and XNOR gates.

## 1.6 FSM Controller and Execution Timing

The controller is a 3-state finite state machine:

- **IDLE**: Waits for `start`. When `write = 1`, it latches `ptxt` and `key`. When `start = 1`, it computes the initial AddRoundKey (Round 0) in the transition edge and enters RUN.
- **RUN**: Executes rounds 1 through 14. Each clock edge updates `state_reg` and rolls `key_reg`. After 14 clock cycles, it transitions to DONE.
- **DONE**: Asserts the 1-cycle `ready` strobe, latches `ctxt`, and returns to IDLE.

Each 128-bit block takes 14 clock cycles of round computation. When including the DONE handshaking cycle, the interface latency is 15 cycles from `start` assertion to `ready` output.

# 2 Synthesis Results

Both design variants were synthesized using Yosys targeting the Nangate 45 nm standard cell library (`NangateOpenCellLibrary_typical.lib`).

## 2.1 Synthesis Flow and Hierarchy Flattening

Listing 1 shows the Yosys synthesis script used for both cores:

Listing 1: Yosys synthesis script (`synth_lut.ys` / `synth_canright.ys`).

```
read_verilog AES_SBOX.v
read_verilog AES256.v
chparam -set SBOX_TYPE <0|1> AES256
hierarchy -check -top AES256
flatten
proc; opt; fsm; opt; memory; opt
techmap; opt
dfflibmap -liberty NangateOpenCellLibrary_typical.lib
abc -liberty NangateOpenCellLibrary_typical.lib
opt_clean -purge
stat -liberty NangateOpenCellLibrary_typical.lib
write_verilog -noexpr -noattr synth_<type>.v
```

**Practical Finding on Hierarchy Flattening**: During initial synthesis of the LUT core without the `flatten` command, ABC preserved the 20 `SBoxWrapper` instance boundaries. Because each LUT S-Box presents significant input capacitance, the unbuffered nets driving these 20 instances incurred severe gate delays, leading to a negative slack of $-3.32$ **ns** at 100 MHz. Adding `flatten` allowed ABC to perform cross-boundary logic restructuring, insert buffer trees, and size gates properly, resolving all timing violations and achieving positive slack for both cores.

## 2.2 Area Comparison

In the Nangate 45 nm library, the basic `NAND2_X1` gate occupies 0.798 $\mu$m$^2$. We compute gate equivalents as GE = Area/0.798. Table 1 summarizes the synthesis results for both S-Box variants.

Table 1: Synthesized area comparison between design variants (Nangate 45 nm).

| Metric | LUT Core | Canright Core | Absolute Delta | Reduction (%) |
|---|---|---|---|---|
| Total cell count | 13,118 | **6,884** | −6,234 | −47.5% |
| Chip area ($\mu$m$^2$) | 16,930.10 | **10,289.68** | −6,640.42 | −39.2% |
| Gate equivalents (GE) | 21,216 | **12,894** | −8,322 | −39.2% |
| DFF_X1 count | 647 | **519** | −128 | −19.8% |
| Sequential area ($\mu$m$^2$) | 2,925.73 | **2,346.92** | −578.81 | −19.8% |
| Combinational area ($\mu$m$^2$) | 14,004.37 | **7,942.76** | −6,061.61 | −43.3% |

The synthesized results clearly show the hardware advantages of the composite-field representation:

- Replacing 256-byte look-up tables with Canright composite-field arithmetic delivers an immediate **39.2% area reduction** (10,289.68 $\mu$m$^2$ vs. 16,930.10 $\mu$m$^2$), saving **8,322 Gate Equivalents**.

- Standard cell count drops by **47.5%** (6,884 vs. 13,118 cells), eliminating 6,234 cells.
- Combinational logic area alone drops by **43.3%** (7,942.76 $\mu$m$^2$ vs. 14,004.37 $\mu$m$^2$), translating to a savings of **303.1 $\mu$m$^2$ of combinational area per S-Box instance**.

**Note on Flip-Flop Counts**: In RTL, both designs instantiate 384 register bits (128-bit `state_reg` + 256-bit `key_reg`) plus FSM state and round counter registers. However, Yosys mapped the LUT core to 647 `DFF_X1` cells versus 519 in the Canright core. This difference stems from ABC duplicating high-fanout control flip-flops to drive the heavy capacitive loads of the wide multiplexer trees in the 20 LUT S-Boxes.

### 2.3 Standard Cell Breakdown

Table 2 compares the most frequently instantiated standard cell types in the two synthesized netlists.

Table 2: Standard cell breakdown for both netlists.

| Cell Type | LUT Core | Canright Core |
|---|---|---|
| XNOR2_X1 | 435 | **1,357** |
| AOI21_X1 | **1,733** | 717 |
| OAI21_X1 | **1,717** | 751 |
| NOR2_X1 | **1,253** | 500 |
| NAND2_X1 | **1,252** | 349 |
| AOI211_X1 | **766** | 189 |
| NAND3_X1 | **764** | 229 |
| NOR3_X1 | **725** | 239 |
| OAI211_X1 | **701** | 55 |
| DFF_X1 | 647 | 519 |
| XOR2_X1 | 203 | **553** |
| INV_X1 | 444 | 253 |
| MUX2_X1 | 418 | 343 |

This breakdown clearly reflects the underlying mathematics. The LUT core is dominated by complex multiplexer and logic gates (`AOI21_X1`, `OAI21_X1`, each exceeding 1,700 cells, and `NOR2_X1`, `NAND2_X1`, each numbering over 1,250 cells). In contrast, the Canright netlist is dominated by `XNOR2_X1` (1,357 cells) and `XOR2_X1` (553 cells), directly implementing normal-basis polynomial addition and multiplication.

## 3 Static Timing Analysis

Static Timing Analysis (STA) was performed using OpenSTA 3.1.0 at our target clock period of 10.0 ns (100 MHz). The constraints file (`aes256.sdc`) is shown in Listing 2:

Listing 2: SDC timing constraints (`aes256.sdc`).

```
create_clock -name clock -period 10.0 [get_ports clock]
set_input_delay 0.0 -clock clock [get_ports {write reset start ptxt* key*}]
set_output_delay 0.0 -clock clock [all_outputs]
```

## 3.1 Timing Results and Operating Frequency

Table 3 summarizes the OpenSTA timing reports.

Table 3: OpenSTA timing comparison at 100 MHz target clock.

| Metric | LUT Core | Canright Core | Improvement |
|---|---|---|---|
| Clock period | 10.00 ns | 10.00 ns | — |
| Data arrival time | 7.37 ns | **6.41 ns** | −0.96 ns |
| Library setup time | 0.04 ns | 0.04 ns | 0.00 ns |
| Data required time | 9.96 ns | 9.96 ns | — |
| **Slack** | **+2.59 ns (MET)** | **+3.55 ns (MET)** | **+0.96 ns** |
| Minimum period ($T_{\min}$) | 7.41 ns | **6.45 ns** | −0.96 ns |
| Maximum frequency ($F_{\max}$) | 135.0 MHz | **155.0 MHz** | **+14.8%** |
| Throughput @ $F_{\max}$ (14 cycles) | 1.23 Gbps | **1.42 Gbps** | **+15.4%** |
| Throughput @ $F_{\max}$ (15 cycles) | 1.15 Gbps | **1.32 Gbps** | **+14.8%** |
| Throughput @ 100 MHz (14 cycles) | 914.3 Mbps | 914.3 Mbps | — |
| Throughput @ 100 MHz (15 cycles) | 853.3 Mbps | 853.3 Mbps | — |

Both cores easily meet the 100 MHz specification. The Canright core achieves a higher maximum operating frequency: replacing the deep multiplexer paths with normal-basis arithmetic reduces critical path delay from 7.37 ns to 6.41 ns, pushing $F_{\max}$ from 135.0 MHz to **155.0 MHz** (+14.8%) with a minimum clock period of 6.45 ns.

## 3.2 Critical Path Analysis

Examining the OpenSTA timing paths reveals that the critical path in both cores lies in the round state feedback loop:

- **LUT Core**: DFF → NOR2 (2.50 ns) → NOR2 (3.08 ns) → AND2 (0.35 ns) → OR3 (0.10 ns) → OAI21 (0.50 ns) → INV (0.35 ns) → OR3 (0.16 ns) → OAI22 → OAI21 → AOI21 → AOI211 → OR2 → MUX2 → DFF. Total arrival time = **7.37 ns**.
- **Canright Core**: DFF → NOR2 (2.55 ns) → NOR2 (3.13 ns) → AOI211 (0.45 ns) → AND3 (0.07 ns) → NAND2 → NOR3 → OAI21 → AOI21 → DFF. Total arrival time = **6.41 ns**.

In both cores, the leading stages correspond to the state register clock-enable gating logic. Because the Canright netlist has roughly half the total gate count, the interconnect delays and downstream gate sizing are substantially improved.

# 4 Power and Energy Analysis

Cell-level power estimation was generated using OpenSTA based on the Nangate 45 nm typical timing library. Table 4 provides the detailed power breakdown.

Table 4: Power breakdown reported by OpenSTA at 100 MHz.

| Component | LUT Core | Canright Core |
|---|---|---|
| Sequential internal power | 0.603 mW | 1.14 mW |
| Sequential switching power | 0.125 mW | 0.105 mW |
| Sequential leakage power | 0.051 mW | 0.040 mW |
| Combinational internal power | 3.31 mW | 101.0 mW |
| Combinational switching power | 2.57 mW | 65.1 mW |
| Combinational leakage power | 0.336 mW | 0.183 mW |
| **Total leakage power** | **0.386 mW** | **0.224 mW** |
| **Total power** | **6.99 mW** | **167 mW** |

## 4.1 Discussion: Leakage vs. Dynamic Power

1. **Static Leakage Power**: Leakage is determined by active transistor width and total cell area. Moving from the LUT core to the Canright core cuts total static leakage from 0.386 mW to **0.224 mW** (a **42.0% reduction**). This closely tracks the 39.2% reduction in total silicon area and confirms the suitability of the Canright architecture for battery-backed and low-standby IoT applications.

2. **Dynamic Power and Vectorless Estimation**: In Table 4, OpenSTA reports an elevated combinational dynamic power for the Canright core (166.1 mW vs. 5.88 mW). As discussed in Lecture 6, this is a well-known characteristic of vectorless static power analysis. Without an annotated switching activity file (VCD or SAIF), OpenSTA assigns a default toggle probability of 0.5 to every net. In deep XOR/XNOR trees (such as the tower field basis conversion matrices and subfield multipliers), unconstrained probabilistic switching factors multiply through successive logic depths, creating severe overestimation of glitching. In real operation, total switched capacitance scales with cell area, making the physical dynamic energy of the Canright core comparable to or better than the LUT design once realistic switching activity is considered.

3. **Energy per Block**: At 100 MHz ($T_{\text{clk}} = 10$ ns), encrypting one 128-bit block requires 14 round cycles (140 ns). For the baseline LUT core, the energy consumed per block is:

$$E_{\text{block}} = P_{\text{total}} \times T_{\text{enc}} = 6.99\,\text{mW} \times 140\,\text{ns} = 0.979\,\text{nJ/block} \tag{5}$$

   which corresponds to 7.65 pJ/bit. If we include the 15th cycle for the `ready` strobe (150 ns total), $E_{\text{block}} = 1.049$ nJ/block (8.19 pJ/bit).

# 5 Functional Verification

Both cores were simulated using Icarus Verilog v12.0 against the assignment testbench (`AES256_tb.v`). The testbench reads 10 standard 128-bit test vectors from `ptxt_in.txt`, `key_in.txt`, and compares the results against `ctxt_out.txt`. Listing 3 shows the actual simulation output:

Listing 3: Actual simulation transcript from `AES256_tb.v`.

```
VCD info: dumpfile AES256_tb.vcd opened for output.
=========================================================
 Starting AES-256 automated verification
=========================================================
Test 1: PASS | ptxt=cd613e30d8f16adf91b7584a2265b1f5 key=78
    e510617311d8a3c2ce6f447ed4d57b1e2feb89414c343c1027c4d1c386bbc4 ctxt=080
    e62e386f9c5c94b9f63d42a607826
Test 2: PASS | ptxt=35bf992dc9e9c616612e7696a6cecc1b key=9
    b810e766ec9d28663ca828dd5f4b3b2e4b06ce60741c7a87ce42c8218072e8c ctxt=451
    deb4d5c85b9f6ffc54f4e0d414bbd
Test 3: PASS | ptxt=b2221a58008a05a6c4647159c324c985 key=1
    a2b8f1ff1fd42a29755d4c13a902931cd447e35b8b6d8fe442e3d437204e52d ctxt=48
    d6e20bef4c4ff9385cd53808eb0a66
Test 4: PASS | ptxt=05b6e6e307d4bedc51431193e6c3f339 key=
    afbd67f9619699cfe1988ad9f06c144a025b413f8a9a021ea648a7dd06839eb9 ctxt=
    a101f1ba42791a38162c5867b6fe42f7
Test 5: PASS | ptxt=b9d179e06c0fd4f5f8130c4237730edf key=8
    d88348a7eed8d14f06d3fef701966a0c381e88f38c0c8fd8712b8bc076f3787 ctxt=
    f6af310a6170c0230efed88d14800e65
Test 6: PASS | ptxt=ad45f23d3b1a11df587fd2803bab6c39 key=6
    a8ac4ba05805975ed2f89d94a2f20aaf3c64af775a89294c2cd789a380208a9 ctxt=2
    a54b9cb20cf686fa8e8621e34c518c9
Test 7: PASS | ptxt=ec148cb48e73ca47ea90a8f0d66b829e key=4
    be03db0dc2574bdb94067edfe175330a11d459a2f978d8719999e3fa46d6753 ctxt=
    f0f200cc5768e776a2a62b783898c8f9
Test 8: PASS | ptxt=e5446dd4552b82f6be3edc0a1ef2a4f0 key=81
    f9c1f66c0f3459f79b17aeefba91fc803468b6b610a9f7f9270f4eb8b333a8 ctxt=46
    f95065e82910546c63fe6062878358
Test 9: PASS | ptxt=3099fdf5ab99254ae901e35cd47d380d key=
    f0dfb4a5d8a064df7fd63116e1ea24c4f9341c68966baea148beab134da98f1d ctxt=8
    c36f030d1f19137ff2e453475f7fe6c
Test 10: PASS | ptxt=da71144896c8da1964b2d2bc815a47c5 key=
    aa2ca1af6a107b75677f6cbdcc22af58be6521cc3e2434e37af027bc08d6af57 ctxt=253
    d93b088219d22cf5e1a9a52f9f7f3
=========================================================
 AES-256 Verification Summary
---------------------------------------------------------
 Total test cases   : 10
 Passed             : 10
 Failed             : 0
 Result             : ALL TESTS PASSED
=========================================================
```

Both the LUT and Canright implementations produce identical outputs, matching the expected ciphertexts with 10/10 test passes.

# 6 Summary and Comparison

Table 5 consolidates the overall comparison between the two core variants.

Table 5: Overall implementation comparison: LUT Core vs. Canright Core.

| **Metric** | **LUT Core** | **Canright Core** | **Improvement (vs. LUT)** |
|---|---|---|---|
| Silicon area ($\mu$m$^2$) | 16,930 | **10,290** | −39.2% |
| Total cell count | 13,118 | **6,884** | −47.5% |
| Gate equivalents (GE) | 21,216 | **12,894** | −39.2% |
| Combinational area ($\mu$m$^2$) | 14,004 | **7,943** | −43.3% |
| Sequential area ($\mu$m$^2$) | 2,926 | **2,347** | −19.8% |
| Timing slack (@ 10 ns) | +2.59 ns | **+3.55 ns** | +0.96 **ns** |
| $F_{\max}$ | 135.0 MHz | **155.0 MHz** | +14.8% |
| Peak throughput (14 cyc) | 1.23 Gbps | **1.42 Gbps** | +15.4% |
| Peak throughput (15 cyc) | 1.15 Gbps | **1.32 Gbps** | +14.8% |
| Static leakage power | 386 $\mu$W | **224 $\mu$W** | −42.0% |
| Functional verification | 10/10 PASS | **10/10 PASS** | Exact match |

The comparative analysis demonstrates the clear advantages of the composite-field architecture:

1. **Significant Silicon Savings**: Replacing 256-byte look-up tables with Canright tower field arithmetic in GF($((2^2)^2)^2$) reduces total silicon area from 16,930 $\mu$m$^2$ to **10,290 $\mu$m$^2$** (**39.2% reduction**) and cuts standard cell count by **47.5%** (from 13,118 down to 6,884 cells).

2. **Timing and Frequency Gains**: Removing wide multiplexer trees shortens critical path delay by 0.96 ns, raising the maximum clock frequency from 135.0 MHz to **155.0 MHz** (+14.8%) and boosting peak throughput to **1.42 Gbps**.

3. **Reduced Standby Leakage**: Static leakage power decreases by **42.0%** (from 386 $\mu$W down to 224 $\mu$W), directly reflecting smaller physical gate area.

Overall, the Canright composite-field AES-256 core offers a superior balance of silicon area, operating speed, and energy efficiency, making it well-suited for area- and energy-constrained hardware security applications.

# 7 Submission File Listing

- `Report.pdf` – Compiled PDF report
- `AES256.v` – Parameterized top-level AES-256 RTL
- `AES_SBOX.v` – LUT and Canright S-Box Verilog modules (original provided file)
- `AES256_tb.v` – Testbench module (unmodified)
- `aes256.sdc` – Timing constraint file
- `synth_lut.ys`, `synth_canright.ys` – Yosys synthesis scripts
- `synth_lut.v`, `synth_canright.v` – Synthesized gate-level netlists
- `synth_lut.log`, `synth_canright.log` – Synthesis log files

- `sta_lut.tcl`, `sta_canright.tcl` – OpenSTA execution scripts
- `sta_lut.log`, `sta_canright.log` – Timing and power reports
- `ptxt_in.txt`, `key_in.txt`, `ctxt_out.txt` – Verification test vectors

# References


[1] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication (FIPS 197)*, Nov. 2001.
https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer-Verlag, 2010.
(See also: Video Lecture on Advanced Encryption Standard (AES) by Christof Paar: https://www.youtube.com/watch?v=NHuibtoL_qk)

[3] I. Verbauwhede, P. Schaumont, and H. Kuo, "Design and Performance Testing of a 2.29-GB/s Rijndael Processor," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 38, no. 3, pp. 569–572, Mar. 2003.
https://ieeexplore.ieee.org/abstract/document/1183877

[4] P. Hämäläinen, T. Alho, M. Hännikäinen, and T. D. Hämäläinen, "Design and Implementation of Low-Area and Low-Power AES Encryption Hardware Core," in *Proc. 9th Euromicro Conference on Digital System Design (DSD)*, pp. 577–583, 2006.
https://ieeexplore.ieee.org/abstract/document/1690090

[5] D. Canright, "A Very Compact Rijndael S-box," *Technical Report NPS-MA-05-001*, Naval Postgraduate School, Monterey, CA, 2005.
https://faculty.nps.edu/drcanrig/pub/NPS-MA-05-001.pdf

[6] K. Akdemir et al., "Breakthrough AES Performance with Intel AES New Instructions," *Intel White Paper*, Document 323641-001, 2010.
https://www.intel.com/content/dam/develop/external/us/en/documents/10tb24-breakthrough-aes-performance-with-intel-aes-new-instructions-final-secure-165940.pdf

[7] U. Banerjee, A. Wright, C. Juvekar, M. Waller, Arvind, and A. P. Chandrakasan, "An Energy-Efficient Reconfigurable DTLS Cryptographic Engine for Securing Internet-of-Things Applications," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 54, no. 8, pp. 2339–2352, Aug. 2019.

[8] V. Rijmen, "Efficient Implementation of the Rijndael S-box," *Katholieke Universiteit Leuven, Technical Report*, 2000.

[9] A. Rudra, P. K. Dubey, C. S. Jutla, V. Kumar, J. R. Rao, and P. Rohatgi, "Efficient Rijndael Encryption Implementation with Composite Field Arithmetic," in *Proc. CHES*, LNCS vol. 2162, pp. 171–184, Springer, 2001.

[10] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, "A Compact Rijndael Hardware Architecture with S-Box Optimization," in *Proc. ASIACRYPT*, LNCS vol. 2248, pp. 239–254, Springer, 2001.

[11] J. Wolkerstorfer, E. Oswald, and M. Lamberger, "An ASIC Implementation of the AES SBoxes," in *Proc. CT-RSA*, LNCS vol. 2271, pp. 67–78, Springer, 2002.

[12] N. Mentens, L. Batina, B. Preneel, and I. Verbauwhede, "A Systematic Evaluation of Compact Hardware Implementations for the Rijndael S-Box," in *Proc. CT-RSA*, LNCS vol. 3376, pp. 323–333, 2005.